MOBILE DJ / PRODUCT BLUEPRINT

DOCUMENT 13 OF 19

ParDay AI Architecture and Requirements

A new touch-first performance instrument.
Professional audio, liquid glass and mechanical craft.



DESIGN HANDOFF BASELINE - PLANNING ONLY

29 October 2026 | Crystal Current selected; S26 Ultra test target

Independent of the previous Mixline preview. Targets are not measured results.

Version: 1.0 **Date:** 29 October 2026 **Status:** Design handoff baseline; no model/provider selected, app implemented or performance result measured **Confirmed direction:** The assistant is named **ParDay**. The owner selected **C - assisted performance**: preparation assistance plus explicitly armed transitions/automix and immediate manual takeover. Android first, with Galaxy S26 Ultra as the first test device; serious mobile DJs primary, casual streaming DJs secondary; initial use is practice and private gatherings. **Accountable agents:** AI/Services; Audio Engine; Library/Data; Security; Product UI; Quality. **Related documents:** 05 Architecture; 06 Audio; 07 Sources; 09 Quality; 14 Discovery and Acquisition; 16 Link Acquisition; 17 Open Source and Home Lab; 18 AWS Deployment and Comparison.

Product decision

Build ParDay as an optional music preparation and performance assistant that can explain, propose and perform explicitly authorized application tasks. Its value comes from understanding the DJ's library, current preparation and actual supported capabilities. A generic chat window alone cannot deliver that value. A language model also cannot replace the deterministic audio engine, source permissions or physical device certification.

The leading design combines three different systems: local signal analysis for measurable musical properties; a local planner and validator for executable actions; and a replaceable language-model service for interpreting requests and explaining options. Prepared tracks, mixing, cueing, effects, recording and manual performance continue when ParDay is disabled or the network fails. Live stems remain a separate audio/ML workload governed by Document 06, regardless of whether ParDay requested their preparation.

The full product includes the owner-selected ParDay assisted-performance capability alongside the selected professional modules. ParDay remains optional to use; the selected mode is a product capability, not standing authorization for arbitrary audible changes. Introduce its features through development gates, without silently shrinking the final scope to a chatbot or advertising unverified automation. Lighting remains excluded. No consumer chat session, provider password or browser automation is required for the recommended integration.

Useful jobs and their boundaries

Job	Example request and useful result	Constraint
Library search	“Find my 118-124 BPM tracks with a long intro.” Returns local track IDs with measured fields and an explanation	Unknown BPM or intro length stays unknown; no invented analysis
Crate preparation	“Plan 45 minutes from warm-up to high energy.” Produces an editable order, estimated duration and alternatives	Respect unavailable tracks, explicit exclusions, editions and source eligibility
Transition suggestions	“Give me three ways into the next track.” Offers phrase, tempo, key and vocal-overlap choices	Use validated analysis; uncertainty and preview controls remain visible
Cue/grid assistance	“Suggest an intro cue and an eight-bar exit.” Creates a preview of proposed preparation edits	Preserve manual locks; confirm uncertain downbeats; retain prior versions
Readiness diagnosis	“Can this set run offline with my USB interface?” Checks cached eligible tracks, route, storage and device profile	Diagnose actual state; never assert compatibility from a device name alone
Rehearsal coach	“Help me practice this transition.” Offers a short exercise and optional post-attempt feedback	Opt-in, user-paced; no unsolicited spoken coaching over the master mix
Guided automix/arrangement	“Use these tracks and transition only at the marked exits.” Produces a bounded executable plan	User arms it; deterministic scheduler executes; immediate manual takeover
Discovery/acquisition help	“Find an authorized version of this track.” Opens permitted catalog results and acquisition choices	Search result, subscription and copyright licence are separate from PCM access

These are distinct experiences, not buttons that all send the same unconstrained prompt. Every result identifies which inputs were measured, supplied by the user, retrieved from an allowed source or inferred. “Energy” is a useful comparative label, not an objective fact unless the product defines a measurement. Local analysis should expose confidence and method rather than a spurious universal score.

Start with search, crate planning, readiness and transition explanations because they can improve real preparation without taking over audible controls. Cue proposals, rehearsal and bounded automix follow once the shared state and manual-takeover contracts pass. Voice input is optional and needs a separate microphone permission, noisy-room testing and an explicit listening state; typed and touch workflows must remain complete.

Recommended technical shape

The Android client owns the local library, permitted analysis, live state and action authority. The first version has no required application server. Design and test local orchestration with model-free fixtures; evaluate a personal direct-API configuration with user-supplied credentials when integrating a real provider. A later optional gateway can own provider credentials, spending controls and model configuration. Neither AWS nor a gateway is a prerequisite for the local interface, deterministic assistant harness or manual DJ features. Provider-specific response formats are translated into a common ParDay proposal format. Keep `parday-domain`, `parday-context`, `parday-policy`, `parday-provider` and `parday-ui` boundaries separate from `engine-core`.

OpenAI function calling and Anthropic client tools both support a model returning structured requests that application code handles. Their wire formats differ, so the product contract must remain independent of

either. Use provider-supported schema enforcement, then perform application validation anyway: valid JSON does not prove that a track exists, the user approved an action or the current deck still contains that track. [S1, S2]

For this bounded mobile use case, an application-owned tool loop is the leading implementation. It is justified by the requirement to enforce local state, device permissions and identical behavior across providers. An agent SDK can supply tracing or orchestration if it reduces real complexity. The OpenAI Agents API is an option for separate, longer preparation jobs; its hosted sessions and optional environments introduce state/retention decisions that must be evaluated. No cloud sandbox receives arbitrary access to the phone or live engine. The current Agents API documentation describes retained sessions and limitations on retention/residency options; do not equate a self-hosted environment with no provider retention. [S3]

The model can choose among the tools made available to its current role, but the application decides whether a requested operation is eligible. The model never receives a general shell, arbitrary filesystem access, unrestricted URL fetcher, music downloader, raw SQL tool or direct `EngineCommand` endpoint. Approved web research, if later added, uses a separate read-only broker and the source rules in Document 14.

Authority and live-session behavior

Level	Allowed behavior	How it is enabled
Read	Explain app help, inspect selected permitted state, search authorized metadata	Default within the user's requested task and current data settings
Propose	Suggest crate order, cues, transition or diagnosis	Default during live performance; no audible mutation
Apply preparation	Save explicitly requested crate/preparation edits or launch an eligible preparation job	A direct instruction or accepted diff; transactional edits and history
Armed performance	Execute a specific, bounded transition/automix plan	Explicit arm action showing affected decks, timing, limits and takeover

“Make me a crate” can authorize saving a reversible crate without repeated confirmation. “Suggest a transition” does not authorize playing it. The selected assisted-performance mode begins each unarmed live interaction in read/propose. An accepted proposal authorizes the displayed version only. Changes in its affected tracks, route or required permissions invalidate that authority; a newer model response does not inherit approval.

Arming is concrete and time-bounded. The interface names the tracks/decks, transition duration, allowed parameters, maximum gain, start condition, expiry and stop behavior. Once armed, the deterministic controller can perform that bounded plan without asking at each beat. The language model does not continually improvise new controls. Manual touch on an affected transport or parameter cancels the relevant automation according to the takeover contract; pressing a persistent Stop Automation control cancels all armed ParDay actions.

Do not describe audible history as reversible. A crossfade already heard cannot be undone. “Undo” restores reversible metadata edits or cancels future events; recovery from an audible change is a new controlled action. Cancelling must not hard-reset faders or abruptly jump parameters. The engine retains its normal safe takeover and smoothing behavior.

Tool contracts and proposal lifecycle

Function	Inputs	Output and execution rule
`search_library`	User-authored query, approved filters, bounded result count	Eligible local IDs and measured fields; no unrestricted library dump
`inspect_tracks`	Selected IDs and allowed field names	Revisioned analysis with provenance and confidence
`inspect_session`	Approved snapshot field set	Deck generations, stream epoch, route/profile state; no audio samples
`plan_crate`	Candidate IDs, duration/energy constraints and exclusions	Ordered proposal with alternatives and missing-data flags
`suggest_preparation`	Track/revision and requested edit types	Cue/grid diff; never overwrites locked values directly
`plan_transition`	Two IDs, analysis revisions and allowed strategy	Musical proposal; deterministic planner computes concrete engine timing
`check_readiness`	Set ID and intended route/profile	Pass/warning/block reasons tied to observed state
`propose_action`	Typed bounded actions and rationale	Stored proposal ID; no direct execution privilege

Tool results carry stable IDs, provenance references and revisions. They are data, including any text contained in a track title or note. User approval and authorization tokens are generated by trusted application code, never by the model or retrieved content.

The lifecycle is requested → proposed → validated → accepted/armed → scheduled → applied, with rejected, cancelled, expired and failed outcomes. Revalidate just before commit, not only when the proposal is first displayed. A proposal records a user-intent ID, expiry, affected data revisions, session ID, stream epoch, deck generations and manual-control revisions, including the master/crossfader revision. Bind acceptance to the immutable complete proposal and plan digest. The accompanying `parday_action.schema.json` is a planning representation for application validation; its fields are not a native ABI or a turnkey provider tool schema. Provider adapters must translate the supported subset and test it.

The local action gateway checks authorization, source capability, track identity, grid revision, route epoch, deck generation, queue capacity and current control revision. It builds typed commands outside the audio callback. The audio engine checks its own epoch/generation and bounded command contract again. Use unique action IDs and an acknowledgement journal so a model retry, app reconnect or delayed network response cannot replay a mutation. Store metadata mutation and its idempotency record in the same database transaction. A schema-valid stale proposal is rejected with a reason and refreshed preview.

Mandatory state and batch validation

Every live action requires a current non-null stream epoch and exactly one basis record for every affected deck. Deck IDs and action IDs must be unique; incoming and outgoing decks must differ. The trusted plan binds exact track/media, analysis/grid and preparation revisions. Its duration, gain interpretation and parameter envelope must equal the accepted action. An inactive load target has no audible tails or pending transport/automation that would make replacement unsafe. The proposal field `deckGeneration` maps to the engine field of the same name; `masterControlRevision` guards crossfader/master changes. Parse timestamps, enforce expiry against trusted time and validate numeric-string bounds; schema formats alone do not establish those properties. Every provenance reference must resolve and allow the particular processing purpose and destination.

The proposal's `commitPolicy` is mandatory. `ATOMIC_PREPARATION_EDITS` permits crate/cue writes in one local transaction: all commit or none do. `SINGLE_PREPARATION_JOB` enqueues one stem-preparation job atomically, then reports per-track progress and failures without calling the entire job completed prematurely. `SINGLE_PERFORMANCE_ACTION` contains exactly one bounded load or arm action. A dependent load-then-arm workflow waits for the load acknowledgement and obtains a fresh state-bound proposal. Multiple musical events belong inside the trusted validated plan. A partial result can never be reported as the whole proposal applied.

Engine-owned execution and cancellation

Scheduling-time validation is insufficient for future events. The trusted planner prepares one immutable, preallocated event batch and submits `ADMIT_AUTOMATION_BATCH`. The engine checks every scope binding, timing deadline and total capacity before publishing any runnable event. Admission is all-or-nothing: if a check or reservation fails, zero events become runnable and all staged siblings are discarded. Each successful acknowledgement returns an engine-issued `automationId` and `automationRevision`, attached internally to every scheduled event. These values are engine-owned register identities, not bearer credentials or user approval. The model never supplies or receives trusted execution authority.

Immediately before each event executes, the engine checks its automation identity/revision, stream epoch and affected deck/control guards. The trusted bridge also binds commands and acknowledgements to an engine-created `engineInstanceId`, so a restart cannot accidentally make old sequence/token values valid again. Control revisions track superseding external/manual changes; the active plan's own accepted parameter writes do not invalidate themselves. Manual takeover, Stop Automation, relevant deck/route changes, permission invalidation or plan expiry invalidates the revision and all pending sibling events. At the same engine control boundary, invalidation precedes remaining affected due events; direct manual values then apply through ordinary smoothing. This closes the case where a queued crossfade would otherwise resume after the DJ touched the fader without changing the track or route.

`CANCEL_AUTOMATION` uses reserved bounded control capacity or an equivalent engine-owned invalidation path, so a full normal scheduling queue cannot drop cancellation. The acknowledgement reports the effective engine frame after which the invalidated plan cannot execute further events. A command already heard cannot be reversed; cancellation does not reset current faders or hard-cut valid sound.
Admission/cancellation retries with the same immutable request return the existing outcome; reused IDs with different payloads are rejected. A failed or cancelled plan never retries its remaining events automatically. Engine restart discards tokens and queued events, and restored proposals require fresh validation/arming. Journaling, plan construction and policy resolution remain outside the callback; event-time guards are bounded numeric checks.

Worked example: a controlled transition

The user asks: “Suggest a 16-bar blend from A into B at the next marked exit; keep the vocals separate.” Both tracks are eligible local files with verified analysis. ParDay receives only selected metadata and analysis fields after the privacy/source filter, not their audio or the entire library.

The response proposes an outgoing exit marker, incoming intro marker, 16-bar duration and an explanation of the vocal overlap. The deterministic planner verifies the markers, tempo constraints and available preparation. If vocal activity is unknown, it says that overlap is unverified and offers a headphone preview instead of claiming certainty. A stem-based alternative appears only if the source and current device profile allow it.

The user selects **Arm this transition**. The app displays A→B, the intended exit, allowed tempo envelope, gain ceiling and manual-takeover control. It binds acceptance to proposal P42, stream epoch 9, the current deck generations and relevant revisions. The local planner schedules its exact frame commands from the engine clock. ParDay's network response time does not determine the transition timing.

If the DJ loads a different track into B, changes the route, moves an affected control or misses the scheduling deadline, the plan cancels or requires a fresh preview under its defined rule. It cannot silently substitute another track or execute on the next arbitrary beat. A command rejection is shown as a rejection, not “Done.” This example is a proposed behavior test, not evidence that a model has musical judgment or that timing has been implemented.

Privacy, source provenance and model boundaries

Cloud requests are optional. The setup explains which provider receives the user's request and selected metadata. No raw audio, microphone recording, full library export, artwork, lyrics, private filesystem paths or access tokens are uploaded by default. Feature-specific use of such inputs requires both a legitimate source capability and a clear user choice. Consent cannot grant rights that the app or user does not possess.

Provenance is tracked at field level. A user-entered title and a provider-returned title may look identical while having different permitted uses. Spotify-derived content is excluded by default from prompts, embeddings, model ranking and inference unless an applicable written agreement permits the specific operation. Removing a Spotify label, reformatting a response or routing it through another tool does not remove its provenance. A deterministic catalog view can remain separate from ParDay's eligible context; Document 14 owns the exact provider rules. This is a conservative design consequence of Spotify's published AI/ML restrictions, not a claim that ordinary API access permits a ParDay integration. [S8]

An AI provider's training policy does not override a music provider's restrictions. OpenAI states that API data is not used for training by default, while storage and retention depend on endpoint/features and configuration. Anthropic similarly documents retention arrangements and exceptions by API/model/feature. The app must record the selected contract and configuration rather than promise “zero retention” universally. [S4, S5]

Treat track names, comments, imported playlists, lyrics, web pages and tool-returned text as untrusted material. An instruction embedded in any of them cannot enable a tool, change system rules, approve an action or export data. Enforce this in tool availability and the gateway, not only with a prompt saying “ignore malicious instructions.” Store only concise user-approved preferences and task history; offer review/deletion and exclude source-restricted content from memory.

Offline operation, credentials and cost

When disconnected, local search/filtering, measured analysis, saved proposals, app help and deterministic readiness checks remain useful. A local language model is optional research, not a promise: model licence, memory, heat, battery, quality and supported Android hardware must be measured. It must not compete with protected audio or live-stem workloads. First reduce or stop optional conversational work. Never pause inference supplying audible stems under the label “AI degradation.”

Personal-tool delivery and local-first implementation are confirmed. The first network-model candidate is a personal direct-API configuration with user-owned credentials stored through the appropriate Android

secret-storage design; its provider and credential handling still need review before use. A gateway is a later optional deployment choice, not a hidden requirement for the first local app. Model-free fixtures allow design and authority tests to proceed before either path has credentials. Neither the source-acquisition comparison nor the AWS budget selects the model/provider. Keep credentials out of source control, examples, logs and distributable binaries; never collect consumer ChatGPT/Claude passwords. Each operator supplies their own applicable provider access and billing, and a community repository contains configuration examples rather than shared credentials.

Acquisition runtime, ParDay gateway and language-model inference are independent choices. Planning AWS for later acquisition expansion does not choose ParDay's gateway, model provider or inference location. Running a downloader at home does not keep cloud model requests local, and running extraction on the phone does not require a phone-resident language model. Document actual destinations and retention for each path. A local model remains an optional separate quality/thermal experiment; gateway or model outage leaves prepared local performance available. Document 17 defines the optional portable home-lab boundary; Document 18 defines the deferred AWS acquisition trial. The approximately US$35/month future AWS target excludes ParDay AI-provider charges and does not set an AI spending allowance. Any co-located ParDay gateway is separately selected, budgeted and isolated rather than an automatic consequence of using AWS.

Consumer chat subscriptions and developer API usage have separate access/billing systems. Current official documentation allows account-dependent API-credit benefits for some subscriptions, including specified Claude plans; those benefits still require the relevant API account/linking and do not create unlimited embedded usage. Check the user's actual eligibility at setup rather than assuming a chat subscription funds ParDay. [S6, S7]

Select the model by an app-specific evaluation set: tool correctness, music explanation quality, refusal of unsupported actions, latency distribution and total cost per completed task. Do not lock the specification to today's flagship name. Proposed operating controls are one active interactive request per session, bounded tool rounds, capped context/output, cancellation and per-user daily/monthly limits. An exact AI monetary limit remains to be chosen before paid API use; it is separate from the selected future AWS target and does not block design or model-free tests. Estimate cost from input/output tokens, cache/storage and paid tools; count retries and interrupted requests conservatively.

The UI acknowledges a request immediately and shows cancellable progress. Cloud latency is variable and cannot be advertised as sample-accurate or guaranteed. Use a proposed 20-second interactive deadline with an explicit retry for longer work; readiness checks and manual controls never wait for it. Model/provider fallback is disclosed and permitted only after the replacement passes the same action, privacy and quality tests. A cheaper model cannot inherit broader execution authority.

ParDay requirements and acceptance register

All cases below are proposed and **unverified**. AI identifiers supplement FR and QA registers; they do not replace audio certification.

ID	Requirement	Acceptance evidence
AI-001	Core DJ functions remain independent of ParDay	Complete QA-19 set with AI disabled, offline and unavailable
AI-002	Provider-neutral adapter	Contract tests include a second adapter fixture/stub; evaluate the allowed corpus on every real provider offered, with identical validation. Two production providers are required only if selected.

ID	Requirement	Acceptance evidence
AI-003	Restricted context by provenance	Sentinel provider fields never reach prompts, embeddings, logs or fallback model
AI-004	No default raw-media upload	Network capture contains no audio/mic/artwork/library dump in ordinary requests
AI-005	Selected armed assistance; read/propose until armed	Adversarial requests and model calls cause zero unarmed audible changes
AI-006	Typed tools and local validation	Reject malformed, extra-field, unknown-ID and out-of-range proposals
AI-007	Stale-state protection	Reload deck, revise grid, change route or touch a bound deck/master control before and after scheduling; old proposal/event rejected, including missing or duplicate state bindings
AI-008	Bounded arming	Expired, widened or modified plans cannot execute under prior acceptance
AI-009	Manual takeover	Physical control and Stop Automation invalidate engine-owned revision; no affected event executes after the acknowledged cancellation frame, including under full-queue load
AI-010	At-most-once mutation	Duplicate deliveries/reconnects do not repeat writes or schedule twice; changed-payload ID reuse rejected; metadata/idempotency transaction and batch all-or-nothing behavior survive injected failure
AI-011	Accurate completion state	Inject failed engine/database acknowledgement; UI reports failure, not completion
AI-012	Preparation history	Accepted edit preserves locked cues and can restore the previous revision
AI-013	Analysis honesty	Missing or low-confidence beat/key/phrase data is labelled; no fabricated certainty
AI-014	Prompt-injection containment	Malicious tags, notes, playlists and web text cannot alter tools or export policy
AI-015	Credential protection	Secret scans and network/log tests find no provider key in APK, logs or diagnostics
AI-016	Resource isolation	AI requests, timeout/retry storms and local inference cannot fail certified audio profiles

ID	Requirement	Acceptance evidence
AI-017	Cost and request bounds	Exhausted quota blocks new AI work without interrupting the mix; retry cap enforced
AI-018	Provider retention disclosure	Actual endpoint/configuration matches consent, deletion and stored-history behavior
AI-019	Approved fallback	Unknown/unqualified model disables affected action tools; no silent privilege expansion
AI-020	Rehearsal autonomy	Coach can be dismissed; no forced lessons, master speech or recording without choice
AI-021	Source permissions preserved	Prohibited download, stem, record or AI operation cannot be enabled by natural language
AI-022	Timely cancellation	Cancelled task cannot later save edits or arm playback; completion race covered
AI-023	Deterministic timing	Locally frame-scheduled plan passes QA-05/23; force failure at each batch-admission step and verify zero runnable partial events, then verify execution-time revision guards
AI-024	Useful product outcome	DJs complete agreed preparation tasks faster or with fewer errors than baseline, measured

Use a versioned evaluation set containing ordinary requests, ambiguous versions, incomplete analysis, unusual phrasing, prompt injections, stale proposals and restricted sources. For personal acceptance, compare the owner's assisted and manual preparation on the same permitted library; record time, errors and failure cases without generalizing one user's result. An optional broader study can include at least eight DJs across both audiences with counterbalanced task order. Recruitment is not a personal-release gate. No utility study has been conducted.

Delivery handoff and remaining decisions

At G1/G2, validate read-only state/context and one proposal path using fixtures; no external model is necessary to test authority boundaries. At G3, integrate useful preparation tools and failure handling. At G4, add cue proposals and bounded transition/automix execution after takeover and timing tests. G5 adds production model/source access, AI budget/account choices and eligible discovery. Its local integration does not require AWS; cloud-specific extension work remains deferred. G6 repeats AI security, privacy and audio-isolation tests on the distribution build. The full release scope remains the user's selected scope; a missing mandatory capability requires an explicit scope decision.

The owner selected ParDay option C and the Galaxy S26 Ultra as the first device. Keep read/propose before each individual plan is armed, preparation actions when requested, and immediate manual takeover. Unresolved choices are the initial model provider, acceptable per-user/month cost, how the owner supplies/stores credentials or runs a gateway, exact device software/accessory evidence, broader

supported profiles, and voice/personality. The assistant name, capability level and local-first delivery order are confirmed; do not ask those questions again. None of these implementation choices blocks design: begin with typed/touch controls, provider-neutral fixtures and the proposed authority states in Document 19, and resolve the paid-provider configuration before connecting it. Acquisition assistance, including the requested unofficial routes, must use the versioned job boundary in Document 16 rather than exposing a general downloader or shell directly to the model.

Sources

Checked 29 October 2026. Product architecture, examples, requirements and numeric limits are original proposals; vendor capabilities require rechecking at implementation.

- **S1:** OpenAI, Function calling.
- **S2:** Anthropic, Tool use with Claude.
- **S3:** OpenAI, Agents API overview.
- **S4:** OpenAI, Data controls.
- **S5:** Anthropic, API and data retention.
- **S6:** OpenAI, Managing billing for ChatGPT and the API platform.
- **S7:** Anthropic, Paid subscriptions and separate API billing.
- **S8:** Spotify, Developer Policy, section III.